

```
In [1]: !pip install opencv-python
```

```
Requirement already satisfied: opencv-python in /usr/local/python/3.12.1/lib/python3.12/site-packages (5.0.0.93)  
Requirement already satisfied: numpy>=2 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from opencv-python) (2.5.2)  
[notice] A new release of pip is available: 26.0.1 -> 26.2.1  
[notice] To update, run: pip install --upgrade pip
```

```
In [2]: !pip install matplotlib scikit-image
```

```
Requirement already satisfied: matplotlib in /usr/local/python/3.12.1/lib/python3.12/site-packages (3.11.1)  
Requirement already satisfied: scikit-image in /usr/local/python/3.12.1/lib/python3.12/site-packages (0.26.0)  
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from matplotlib) (1.3.3)  
Requirement already satisfied: cyclor>=0.10 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from matplotlib) (0.12.1)  
Requirement already satisfied: fonttools>=4.28.2 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from matplotlib) (4.64.0)  
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from matplotlib) (1.5.1)  
Requirement already satisfied: numpy>=1.25 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from matplotlib) (2.5.2)  
Requirement already satisfied: packaging>=20.0 in /home/codespace/.local/lib/python3.12/site-packages (from matplotlib) (26.0)  
Requirement already satisfied: pillow>=9 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from matplotlib) (12.3.0)  
Requirement already satisfied: pyparsing>=3 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from matplotlib) (3.3.2)  
Requirement already satisfied: python-dateutil>=2.7 in /home/codespace/.local/lib/python3.12/site-packages (from matplotlib) (2.9.0.post0)  
Requirement already satisfied: scipy>=1.11.4 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from scikit-image) (1.18.1)  
Requirement already satisfied: networkx>=3.0 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from scikit-image) (3.6.1)  
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from scikit-image) (2.37.4)  
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from scikit-image) (2026.8.23)  
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/python/3.12.1/lib/python3.12/site-packages (from scikit-image) (0.5)  
Requirement already satisfied: six>=1.5 in /home/codespace/.local/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)  
[notice] A new release of pip is available: 26.0.1 -> 26.2.1  
[notice] To update, run: pip install --upgrade pip
```

```
In [3]: from skimage.feature import hog
```

```
In [4]: import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
  
from skimage.feature import hog
```

```
print("Libraries imported successfully.")
```

Libraries imported successfully.

```
In [5]: # Load the image
image = cv2.imread("image1.jpeg")

# Check whether image is loaded
if image is None:
    print("Error: image1.jpeg not found.")
else:
    print("Image loaded successfully.")
    print("Image shape:", image.shape)

# Convert BGR to RGB for displaying with matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display image
plt.figure(figsize=(8, 6))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Image loaded successfully.

Image shape: (626, 475, 3)

Original Image



```
In [6]: # Convert image to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

print("Grayscale image shape:", gray_image.shape)

# Display grayscale image
plt.figure(figsize=(8, 6))
plt.imshow(gray_image, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()
```

Grayscale image shape: (626, 475)

Grayscale Image



Observation

The original image was successfully converted into grayscale. Grayscale conversion reduces the image to a single intensity channel, which makes feature extraction easier and computationally efficient.

```
In [7]: # Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and compute descriptors
```

```
keypoints, descriptors = sift.detectAndCompute(gray_image, None)

print("Number of SIFT keypoints:", len(keypoints))

if descriptors is not None:
    print("SIFT descriptor shape:", descriptors.shape)
else:
    print("No descriptors found.")
```

Number of SIFT keypoints: 1740
SIFT descriptor shape: (1740, 128)

Explanation

SIFT detects distinctive points such as corners, edges, and textured regions. For every detected keypoint, SIFT generates a numerical descriptor that represents the local image region.

```
In [8]: # Draw SIFT keypoints on the image
sift_image = cv2.drawKeypoints(
    image_rgb,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

# Display keypoints
plt.figure(figsize=(10, 7))
plt.imshow(sift_image)
plt.title("SIFT Keypoints")
plt.axis("off")
plt.show()
```

SIFT Keypoints



Observation

The detected SIFT keypoints are shown on the image. Keypoints are mainly concentrated around distinctive regions such as corners, edges, and textured areas.

```
In [9]: from skimage.feature import hog  
  
print("HOG imported successfully.")
```

HOG imported successfully.

```
In [10]: # Extract HOG features  
hog_features, hog_image = hog(  
    gray_image,  
    orientations=9,  
    pixels_per_cell=(8, 8),  
    cells_per_block=(2, 2),  
    block_norm="L2-Hys",
```

```
        visualize=True  
    )  
  
    print("HOG feature extraction completed.")  
    print("HOG feature vector length:", len(hog_features))
```

HOG feature extraction completed.

HOG feature vector length: 160776

HOG Parameters

- Orientations = 9
- Pixels per cell = (8, 8)
- Cells per block = (2, 2)
- Block normalization = L2-Hys

These parameters describe the gradient orientations and edge structure of the image.

```
In [11]: # Display HOG visualization  
plt.figure(figsize=(10, 7))  
plt.imshow(hog_image, cmap="gray")  
plt.title("HOG Descriptor Visualization")  
plt.axis("off")  
plt.show()
```

HOG Descriptor Visualization



Observation

The HOG visualization shows the gradient orientation patterns present in different regions of the image. Strong edges and shape boundaries are represented prominently.

Comparison between SIFT and HOG

Parameter	SIFT	HOG
Feature type	Local features	Shape and edge features
Scale invariance	Yes	Limited
Rotation invariance	Yes	Limited
Feature representation	Keypoints and descriptors	Gradient histograms

Parameter	SIFT	HOG
Robustness	High	Moderate
Computational complexity	Relatively high	Generally lower
Main applications	Image matching and recognition	Object and pedestrian detection

Conclusion

SIFT is more suitable for local feature matching and is robust to scale and rotation changes. HOG is computationally simpler and is especially useful for representing object shape and edge information.

```
In [12]: # Create a second similar image by resizing the original image
height, width = gray_image.shape

image2 = cv2.resize(
    gray_image,
    (int(width * 0.8), int(height * 0.8))
)

print("Image 1 size:", gray_image.shape)
print("Image 2 size:", image2.shape)

# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and descriptors
kp1, des1 = sift.detectAndCompute(gray_image, None)
kp2, des2 = sift.detectAndCompute(image2, None)

print("Keypoints in Image 1:", len(kp1))
print("Keypoints in Image 2:", len(kp2))
```

```
Image 1 size: (626, 475)
Image 2 size: (500, 380)
Keypoints in Image 1: 1740
Keypoints in Image 2: 1377
```

```
In [13]: # Create Brute Force Matcher
bf = cv2.BFMatcher()

# Find two nearest matches
matches = bf.knnMatch(des1, des2, k=2)

# Apply Lowe's ratio test
good_matches = []

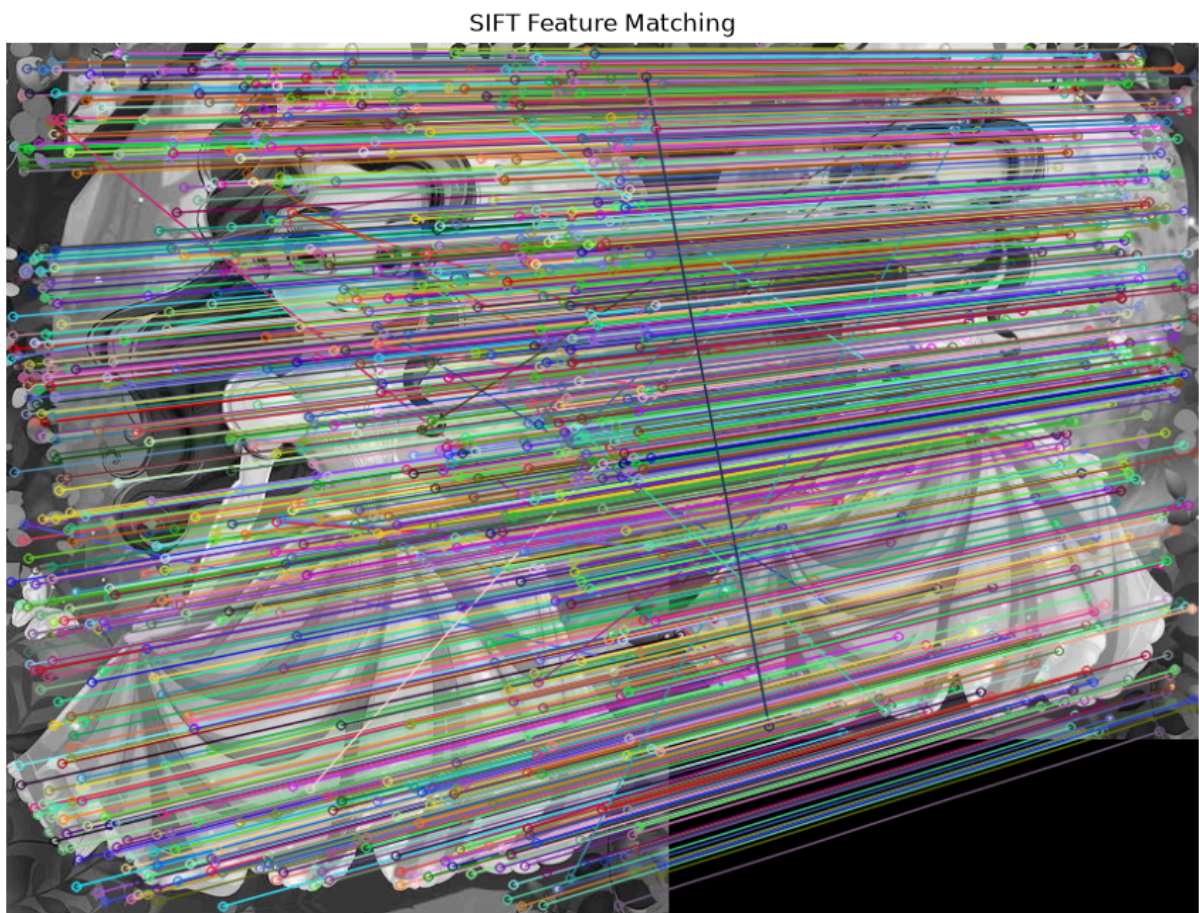
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)
```

```
print("Total matches:", len(matches))  
print("Good matches:", len(good_matches))
```

Total matches: 1740

Good matches: 959

```
In [14]: # Draw good matches  
matched_image = cv2.drawMatches(  
    gray_image,  
    kp1,  
    image2,  
    kp2,  
    good_matches,  
    None,  
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS  
)  
  
# Display matches  
plt.figure(figsize=(16, 8))  
plt.imshow(matched_image, cmap="gray")  
plt.title("SIFT Feature Matching")  
plt.axis("off")  
plt.show()
```



Observation

SIFT successfully detected corresponding features between the original image and the resized version. The matched keypoints demonstrate the ability of SIFT to recognize similar local features even when the image scale is changed.

Strengths and Limitations of SIFT and HOG

SIFT Strengths

1. Scale invariant.
2. Rotation invariant.
3. Robust to moderate illumination changes.
4. Effective for local feature matching.
5. Useful for object recognition and image stitching.

SIFT Limitations

1. Computationally more expensive.
2. Requires more memory for large numbers of descriptors.
3. Can be slower when processing many images.

HOG Strengths

1. Simple and relatively computationally efficient.
2. Captures shape and edge information effectively.
3. Useful for pedestrian detection.
4. Works well with traditional machine-learning classifiers.

HOG Limitations

1. Limited scale invariance.
2. Limited rotation invariance.
3. Sensitive to major changes in object orientation.
4. Mainly represents shape and edge information.

Overall Comparison

SIFT is better for image matching and local feature recognition, while HOG is better for shape-based object detection.

Observations

1. SIFT successfully detected distinctive keypoints in the image.
2. SIFT descriptors provided numerical representations of local image regions.
3. HOG extracted gradient and edge information from the image.
4. HOG visualization clearly represented the shape and edge structure.
5. SIFT was able to match features between the original and resized images.
6. SIFT showed better robustness to scale and rotation changes.
7. HOG was computationally simpler and effective for shape-based analysis.

Applications

Applications of SIFT

- Image matching
- Object recognition
- Image stitching
- Image retrieval
- Object tracking
- Feature correspondence

Applications of HOG

- Pedestrian detection
- Human detection
- Vehicle detection
- Object recognition
- Shape analysis

Result

The SIFT and HOG feature extraction techniques were successfully implemented on the given image. SIFT detected distinctive local features and successfully matched features between similar images. HOG represented the gradient orientation and shape information of the image. SIFT is more suitable for scale- and rotation-invariant feature matching, whereas HOG is useful for shape-based object detection.